

" به نام خدا "



دانشکده فنی مهندسی

رشته مهندسی کامپیوتر

گرایش هوش مصنوعی و نرم افزار

عنوان :

گزارش دوم پروژه نهایی مباحث ویژه

دانشجویان :

محمدرضا باقرنژادمیری، یاسمن دادفر، کیارش رحمانی، معصومه مختاری

استاد :

دکتر مهدیه قاسمی

فهرست

1. مستندات فنی.....	5
MLflow.....	6
1. مقدمه.....	6
2. تاریخچه MLflow.....	6
3. معماری MLflow.....	7
3.1 MLflow Tracking.....	7
3.2 MLflow Projects.....	7
3.3 MLflow Models.....	8
3.4 MLflow Model Registry.....	8
4. کاربردهای MLflow.....	8
6. نصب MLflow.....	9
7. راه اندازی رابط کاربری MLflow.....	9
8. مثال کامل پایتون.....	9
9. اتصال MLflow به SQLite.....	10
10. مزایا و معایب MLflow.....	11
11. جمع بندی.....	11
ETL Pipeline در سامانه های یادگیری ماشین و MLOps.....	12
مقدمه.....	12
تاریخچه ETL.....	12
معماری ETL.....	12
ETL در پروژه های یادگیری ماشین.....	15
مفهوم Idempotency در ETL.....	15
ETL Pipeline در پروژه حاضر.....	16
مزایای ETL.....	16
معایب ETL.....	17
جمع بندی.....	17

17	 Data-Centric AI و نقش آن در بهبود سیستم‌های یادگیری ماشین
17	 مقدمه
18	 تاریخچه Data-Centric AI
18	 تعریف Data-Centric AI
19	 اهمیت داده در یادگیری ماشین
19	 اجزای اصلی Data-Centric AI
21	 Data Augmentation
22	 Class Imbalance
23	 مزایای Data-Centric AI
24	 Data-Centric AI در پروژه حاضر
24	 تفاوت Data-Centric و Model-Centric
25	 جمع‌بندی
27	 2. صورت‌جلسات
27	 1-2 صورت‌جلسه هماهنگی پروژه MLOps (جلسه سوم- هفته دوم)

1. مستندات فنی

MLflow

1. مقدمه

MLflow یکی از محبوب‌ترین پلتفرم‌های متن‌باز برای مدیریت چرخه عمر مدل‌های یادگیری ماشین (Machine Learning Lifecycle) است. این ابزار توسط شرکت Databricks توسعه داده شد و هدف آن حل مشکلاتی مانند مدیریت آزمایش‌ها، ذخیره مدل‌ها، نسخه‌بندی مدل‌ها و استقرار مدل‌ها است.

در پروژه‌های یادگیری ماشین، معمولاً تعداد زیادی آزمایش مختلف انجام می‌شود. پژوهشگران و مهندسان داده دائماً پارامترهای مدل، داده‌های آموزشی و الگوریتم‌ها را تغییر می‌دهند. بدون وجود یک سیستم مدیریت مناسب، پیگیری این تغییرات تقریباً غیرممکن خواهد شد. MLflow دقیقاً برای حل این مشکل طراحی شده است.

امروزه MLflow به عنوان یکی از مهم‌ترین ابزارهای MLOps شناخته می‌شود و در بسیاری از شرکت‌های بزرگ فناوری برای مدیریت مدل‌های هوش مصنوعی مورد استفاده قرار می‌گیرد.

2. تاریخچه MLflow

MLflow در سال 2018 توسط Databricks معرفی شد.

پیش از ظهور MLflow، هر تیم یادگیری ماشین مجبور بود ابزارهای مختلفی را برای ذخیره نتایج آزمایش‌ها، مدیریت مدل‌ها و استقرار آن‌ها توسعه دهد. این مسئله باعث افزایش هزینه و پیچیدگی پروژه‌ها می‌شد.

Databricks با معرفی MLflow تلاش کرد یک چارچوب استاندارد و مستقل از کتابخانه ایجاد کند که بتواند با:

- TensorFlow
- PyTorch
- Scikit-Learn
- XGBoost
- LightGBM

و بسیاری از ابزارهای دیگر کار کند.

به دلیل متن باز بودن و سادگی استفاده، MLflow به سرعت به یکی از استانداردهای صنعت تبدیل شد.

3. معماری MLflow

MLflow از چهار بخش اصلی تشکیل شده است:

MLflow Tracking 3.1

برای ثبت:

- پارامترها (Parameters)
- معیارها (Metrics)
- مدل ها (Artifacts)
- فایل ها

استفاده می شود.

مثال:

```
mlflow.log_param("learning_rate", 0.001)
```

```
mlflow.log_metric("accuracy", 0.92)
```

MLflow Projects 3.2

امکان بسته بندی پروژه ها را فراهم می کند.

هدف آن تضمین Reproducibility است.

به کمک فایل:

MLproject

تعریف می شود.

MLflow Models 3.3

برای ذخیره مدل‌ها استفاده می‌شود.

مثال:

```
mlflow.sklearn.log_model(model, "model")
```

MLflow Model Registry 3.4

برای مدیریت نسخه‌های مختلف مدل استفاده می‌شود.

مدل‌ها می‌توانند در مراحل زیر قرار گیرند:

- None
- Staging
- Production
- Archived

4. کاربردهای MLflow

مهم‌ترین کاربردهای MLflow عبارت‌اند از:

- ثبت آزمایش‌ها

تمامی Hyperparameter ها ذخیره می‌شوند.

- مقایسه مدل‌ها

چندین مدل به راحتی قابل مقایسه هستند.

- نسخه‌بندی مدل

نسخه‌های مختلف مدل نگهداری می‌شوند.

- استقرار مدل

مدل می‌تواند به محیط Production منتقل شود.

- همکاری تیمی

اعضای تیم به اطلاعات یکسان دسترسی دارند.

6. نصب MLflow

```
pip install mlflow
```

بررسی نصب:

```
mlflow --version
```

7. راه‌اندازی رابط کاربری MLflow

```
mlflow ui
```

به صورت پیش‌فرض:

```
http://localhost:5000
```

در مرورگر باز می‌شود.

8. مثال کامل پایتون

```
import mlflow
```

```
with mlflow.start_run():
```

```
    mlflow.log_param(  
        "learning_rate",  
        0.001  
    )
```

```
mlflow.log_param(  
    "epochs",  
    10  
)
```

```
mlflow.log_metric(  
    "accuracy",  
    0.93  
)
```

```
mlflow.log_metric(  
    "loss",  
    0.15  
)
```

9. اتصال MLflow به SQLite

نمونه کد:

```
import mlflow
```

```
mlflow.set_tracking_uri(  
    "sqlite:///mlflow.db"  
)
```

اجرای سرور:

```
mlflow server \  
--backend-store-uri sqlite:///mlflow.db
```

مزایا:

- ذخیره دائمی اطلاعات

- پشتیبانی بهتر از چند کاربر
- مدیریت آسان تر Metadata
- قابلیت استفاده از Model Registry

10. مزایا و معایب MLflow

مزایا

- متن باز
- یادگیری آسان
- پشتیبانی از اکثر فریمورک ها
- رابط کاربری مناسب
- نسخه بندی مدل ها
- مناسب برای MLOps

معایب

- امکانات کمتر نسبت به برخی پلتفرم های تجاری
- نیاز به پیکر بندی بیشتر در پروژه های بزرگ
- Registry پیشرفته نیازمند Backend Database است

11 جمع بندی

MLflow یکی از مهم ترین ابزارهای اکوسیستم MLOps محسوب می شود که امکان مدیریت کامل چرخه عمر مدل های یادگیری ماشین را فراهم می کند. در پروژه حاضر از MLflow برای ثبت آزمایش ها، ذخیره مدل ها، مدیریت نسخه های مدل و استفاده از SQLite به عنوان Backend مرکزی استفاده شد. این ابزار باعث افزایش Reproducibility، قابلیت مقایسه آزمایش ها و مدیریت بهتر مدل ها در محیط های تیمی شده است.

ETL Pipeline در سامانه‌های یادگیری ماشین و MLOps

مقدمه

در پروژه‌های داده‌محور و سامانه‌های یادگیری ماشین، کیفیت داده‌ها تأثیر مستقیمی بر عملکرد مدل دارد. حتی پیشرفته‌ترین الگوریتم‌های هوش مصنوعی نیز در صورت دریافت داده‌های نامناسب، خروجی ضعیفی تولید خواهند کرد. به همین دلیل در صنعت داده، پیش از آموزش مدل، فرآیندی برای جمع‌آوری، پاک‌سازی، تبدیل و آماده‌سازی داده‌ها اجرا می‌شود که با نام ETL شناخته می‌شود.

ETL مخفف سه واژه Extract، Transform و Load است. این فرآیند یکی از بنیادی‌ترین بخش‌های مهندسی داده و MLOps محسوب می‌شود و وظیفه آن آماده‌سازی داده‌های خام برای استفاده در مدل‌های یادگیری ماشین است. در پروژه حاضر نیز یک ETL Pipeline طراحی و پیاده‌سازی شد تا داده‌ها به صورت خودکار از مرحله دریافت تا مرحله آماده‌سازی برای آموزش مدل مدیریت شوند.

تاریخچه ETL

مفهوم ETL نخستین بار در دهه 1970 و همزمان با ظهور Data Warehouse ها مطرح شد. سازمان‌ها نیاز داشتند داده‌های موجود در سامانه‌های مختلف را جمع‌آوری کرده و در یک پایگاه داده مرکزی ذخیره کنند.

در دهه 1990 با رشد سیستم‌های اطلاعاتی، ابزارهای تجاری متعددی برای ETL توسعه یافتند. بعدها با ظهور Big Data و یادگیری ماشین، ETL از یک ابزار سازمانی به یکی از اجزای کلیدی سیستم‌های داده‌محور تبدیل شد.

امروزه تقریباً تمام شرکت‌های بزرگ فناوری از ETL برای مدیریت داده‌های خود استفاده می‌کنند. شرکت‌هایی مانند گوگل، آمازون، مایکروسافت و نتفلیکس روزانه میلیون‌ها رکورد داده را از طریق Pipeline های ETL پردازش می‌کنند.

معماری ETL

فرآیند ETL از سه مرحله اصلی تشکیل شده است:

1. استخراج EXTRACT

در این مرحله داده‌ها از منابع مختلف جمع‌آوری می‌شوند.

منابع داده می‌توانند شامل موارد زیر باشند:

- فایل‌های CSV
- فایل‌های JSON
- پایگاه داده SQL
- API ها
- داده‌های وب

نمونه کد استخراج فایل: CSV:

```
import pandas as pd
```

```
data = pd.read_csv("dataset.csv")
```

نمونه دریافت داده از: API:

```
import requests
```

```
response = requests.get(url)
```

```
data = response.json()
```

هدف این مرحله صرفاً دریافت داده است و هیچ تغییری روی داده اعمال نمی‌شود.

TRANSFORM.2

این مهم‌ترین بخش ETL محسوب می‌شود.

در این مرحله عملیات مختلفی روی داده انجام می‌شود:

- حذف داده‌های ناقص
- حذف داده‌های تکراری

- استانداردسازی داده‌ها
- حذف نویز
- شناسایی داده‌های پرت
- مهندسی ویژگی‌ها
- داده‌سازی (Augmentation)

در پروژه حاضر نیز عملیات حذف داده‌های پرت و تولید نمونه‌های مصنوعی انجام شده است.

نمونه حذف مقادیر خالی:

```
data = data.dropna()
```

نمونه حذف داده‌های تکراری:

```
data = data.drop_duplicates()
```

نمونه استانداردسازی متن:

```
data["text"] = data["text"].str.lower()
```

نمونه تولید ویژگی جدید:

```
data["length"] = data["text"].apply(len)
```

LOAD.3

پس از آماده شدن داده‌ها، خروجی در محل مورد نظر ذخیره می‌شود.

مقاصد رایج ذخیره‌سازی:

- فایل CSV
- پایگاه داده
- Data Warehouse
- Feature Store

• Data Lake

نمونه ذخیره فایل:

```
data.to_csv("processed_data.csv", index=False)
```

ETL در پروژه‌های یادگیری ماشین

در پروژه‌های Machine Learning ، ETL نقش بسیار مهمی دارد زیرا مدل تنها به اندازه کیفیت داده‌های آموزشی خود خوب عمل می‌کند.

مزایای ETL در یادگیری ماشین:

- افزایش کیفیت داده
- کاهش خطاهای آموزشی
- افزایش دقت مدل
- خودکارسازی پردازش داده
- افزایش قابلیت بازتولید آزمایش‌ها

در MLOps معمولاً ETL بخشی از Pipeline کلی آموزش مدل است.

مفهوم Idempotency در ETL

یکی از ویژگی‌های مهم ETL های صنعتی، Idempotency است.

یک Pipeline زمانی Idempotent است که چندین بار اجرای آن، نتیجه نهایی یکسانی تولید کند.

به عنوان مثال اگر Pipeline امروز و فردا روی یک داده ثابت اجرا شود، خروجی دقیقاً یکسان خواهد بود.

این ویژگی در MLOps اهمیت زیادی دارد زیرا باعث می‌شود آزمایش‌ها قابل تکرار و قابل اعتماد باشند.

نمونه ساده:

```
cleaned_data = raw_data.drop_duplicates()
```

اجرای چندباره این دستور نتیجه متفاوتی تولید نمی‌کند.

ETL Pipeline در پروژه حاضر

در این پروژه فایل ingest.py به عنوان نقطه ورود Pipeline عمل می‌کند.

وظایف اصلی این Pipeline عبارت‌اند از:

1. دریافت داده‌های خام
 2. اجرای مراحل پاک‌سازی
 3. حذف داده‌های نامناسب
 4. انجام Data Augmentation
 5. تولید داده آماده آموزش
 6. ذخیره خروجی برای مرحله آموزش مدل
- همچنین ساختار پروژه به گونه‌ای طراحی شده که هر مرحله به صورت مستقل اجرا شود و مراحل مختلف به یکدیگر وابستگی کنترل‌شده داشته باشند.

مزایای ETL

- خودکارسازی پردازش داده
- کاهش خطاهای انسانی
- بهبود کیفیت داده
- افزایش سرعت توسعه
- مناسب برای پروژه‌های بزرگ
- قابلیت ادغام با MLOps

معایب ETL

- نیازمند طراحی اولیه مناسب
- پیچیدگی بیشتر نسبت به اسکریپت‌های ساده
- افزایش زمان توسعه در مراحل ابتدایی
- نیاز به پایش و نگهداری مداوم

جمع‌بندی

ETL یکی از مهم‌ترین اجزای زیرساخت داده در پروژه‌های یادگیری ماشین و MLOps است. این فرآیند امکان استخراج، پاک‌سازی، تبدیل و ذخیره‌سازی داده‌ها را به صورت ساختاریافته فراهم می‌کند. در پروژه حاضر از ETL برای مدیریت چرخه داده، حذف داده‌های نامناسب، تولید داده‌های بهبود یافته و آماده‌سازی مجموعه داده جهت آموزش مدل استفاده شد. استفاده از ETL باعث افزایش کیفیت داده‌ها، قابلیت بازتولید آزمایش‌ها و پایداری Pipeline آموزشی گردید.

Data-Centric AI و نقش آن در بهبود سیستم‌های یادگیری ماشین

مقدمه

در سال‌های اخیر بخش عمده‌ای از تحقیقات هوش مصنوعی بر توسعه مدل‌های پیچیده‌تر، شبکه‌های عمیق‌تر و افزایش تعداد پارامترها متمرکز بوده است. با این حال پژوهشگران به تدریج متوجه شدند که در بسیاری از مسائل، کیفیت داده‌ها تأثیر بیشتری از پیچیدگی مدل بر عملکرد نهایی سیستم دارد. این دیدگاه جدید با عنوان Data-Centric AI شناخته می‌شود.

Data-Centric AI رویکردی است که به جای تمرکز اصلی بر طراحی مدل‌های پیچیده‌تر، بر بهبود کیفیت داده‌ها، اصلاح برچسب‌ها، حذف نمونه‌های نامناسب و افزایش کیفیت مجموعه آموزشی تمرکز می‌کند. هدف این رویکرد ساخت داده‌های بهتر برای آموزش مدل‌های هوشمند است.

در پروژه حاضر نیز از مفاهیم Data-Centric AI برای پاک‌سازی داده‌ها، حذف داده‌های پرت و تولید نمونه‌های جدید استفاده شده است تا عملکرد مدل بدون افزایش پیچیدگی معماری بهبود یابد.

تاریخچه Data-Centric AI

در سال‌های اولیه یادگیری ماشین، بیشتر پژوهش‌ها روی الگوریتم‌ها و مدل‌ها متمرکز بودند. تصور غالب این بود که هرچه مدل پیچیده‌تر باشد، نتایج بهتری حاصل خواهد شد.

با ظهور یادگیری عمیق و مدل‌های عظیم، هزینه آموزش مدل‌ها به شدت افزایش یافت. در همین زمان پژوهشگرانی مانند Andrew Ng مطرح کردند که بسیاری از پروژه‌ها به جای نیاز به مدل‌های جدید، به داده‌های باکیفیت‌تر نیاز دارند.

اندرو انگ در سال 2021 مفهوم Data-Centric AI را به صورت گسترده مطرح کرد و بیان نمود که در بسیاری از کاربردهای صنعتی، بهبود داده‌ها می‌تواند تأثیری بسیار بیشتر از تغییر مدل داشته باشد.

امروزه Data-Centric AI یکی از مهم‌ترین موضوعات حوزه MLOps، یادگیری ماشین صنعتی و مهندسی داده محسوب می‌شود.

تعریف Data-Centric AI

در رویکرد سنتی:

داده ثابت → مدل تغییر می‌کند

در رویکرد Data-Centric:

مدل تقریباً ثابت → داده بهبود می‌یابد

هدف اصلی این رویکرد پاسخ به این سؤال است:

«چگونه می‌توان بدون افزایش پیچیدگی مدل، عملکرد سیستم را بهبود داد؟»

پاسخ:

«با بهبود کیفیت داده‌های آموزشی»

اهمیت داده در یادگیری ماشین

یک قانون معروف در علم داده وجود دارد:

Garbage In = Garbage Out

یعنی اگر داده‌های ورودی بی‌کیفیت باشند، حتی بهترین الگوریتم‌ها نیز خروجی مناسبی تولید نخواهند کرد.

مشکلات رایج داده‌ها عبارت‌اند از:

- داده‌های ناقص
- داده‌های تکراری
- برچسب‌گذاری اشتباه
- داده‌های نویزی
- داده‌های پرت
- عدم توازن کلاس‌ها
- کمبود نمونه آموزشی

Data-Centric AI تلاش می‌کند این مشکلات را برطرف کند.

اجزای اصلی Data-Centric AI

1. Data Cleaning

اولین مرحله پاک‌سازی داده‌ها است.

در این مرحله موارد زیر اصلاح می‌شوند:

- مقادیر خالی
- رکوردهای تکراری
- داده‌های غیرمنطقی
- داده‌های خراب

نمونه کد:

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
```

```
df = df.dropna()
```

```
df = df.drop_duplicates()
```

Label Quality Improvement.2

یکی از مهم‌ترین عوامل کاهش دقت مدل، برچسب‌های اشتباه است.

مثال:

اگر یک متن مثبت به اشتباه به عنوان منفی برچسب بخورد، مدل الگوی اشتباهی یاد می‌گیرد.

در Data-Centric AI بخشی از زمان پروژه صرف بررسی و اصلاح برچسب‌ها می‌شود.

نمونه بررسی برچسب‌ها:

```
print(df["label"].value_counts())
```

Outlier Detection .3

داده‌های پرت می‌توانند باعث کاهش عملکرد مدل شوند.

Outlier به نمونه‌ای گفته می‌شود که تفاوت زیادی با سایر داده‌ها دارد.

مثال:

- متن بسیار کوتاه

- متن بسیار بلند

- داده‌های دارای نویز زیاد

شناسایی داده‌های پرت:

```
import numpy as np
```

```
mean = np.mean(values)
```

```
std = np.std(values)
```

```
outliers = [
```

```
    x for x in values
```

```
    if abs(x - mean) > 3 * std
```

```
]
```

Data Augmentation

یکی از مهم‌ترین تکنیک‌های Data-Centric AI تولید داده‌های مصنوعی است.

گاهی حجم داده‌های آموزشی کم است و مدل نمی‌تواند الگوها را به خوبی یاد بگیرد.

در این شرایط از Data Augmentation استفاده می‌شود.

هدف:

تولید نمونه‌های جدید بر اساس داده‌های موجود.

مثال در NLP

جمله اصلی:

I love this movie

نمونه تولیدشده:

I really like this movie

یا

This movie is amazing

نمونه ساده در پایتون:

```
from nltk.corpus import wordnet
```

```
def synonym(word):
```

```
    syns = wordnet.synsets(word)
```

```
    if syns:
```

```
        return syns[0].lemmas()[0].name()
```

```
    return word
```

در پروژه حاضر نیز تولید نمونه‌های مصنوعی برای افزایش پایداری مدل انجام شده است.

Class Imbalance

یکی از مشکلات رایج داده‌ها عدم توازن کلاس‌ها است.

مثال:

Positive : 9000

Negative : 1000

مدل ممکن است همیشه کلاس Positive را پیش‌بینی کند.

برای حل این مشکل از روش‌هایی مانند:

- Oversampling
- Undersampling
- Synthetic Data Generation

استفاده می‌شود.

نمونه:

```
from sklearn.utils import resample
```

مزایای Data-Centric AI

- افزایش دقت مدل
- کیفیت بهتر داده‌ها مستقیماً دقت را افزایش می‌دهد.
- کاهش هزینه محاسباتی
- نیازی به آموزش مدل‌های بزرگ‌تر نیست.
- افزایش قابلیت تعمیم
- مدل روی داده‌های واقعی عملکرد بهتری خواهد داشت.
- کاهش Overfitting
- داده‌های باکیفیت باعث یادگیری پایدارتر می‌شوند.
- مناسب برای پروژه‌های صنعتی
- در محیط‌های واقعی معمولاً بهبود داده مؤثرتر از تغییر مدل است.

Data-Centric AI در پروژه حاضر

در هفته دوم پروژه، تیم بر بهبود کیفیت داده‌ها تمرکز کرد.

اقدامات انجام‌شده شامل:

- پاک‌سازی داده‌ها
- حذف داده‌های پرت
- اصلاح جریان پردازش داده
- تولید داده‌های مصنوعی
- جلوگیری از Data Leakage
- مدیریت داده‌ها در Pipeline

این اقدامات باعث شد داده‌های آموزشی کیفیت بالاتری داشته باشند و مدل بتواند روی داده‌های قابل اعتمادتر آموزش ببیند.

تفاوت Model-Centric و Data-Centric

Model-Centric AI	Data-Centric AI
تمرکز روی مدل	تمرکز روی داده
افزایش پارامترها	افزایش کیفیت داده
هزینه محاسباتی بیشتر	هزینه کمتر
نیازمند سخت‌افزار قوی	نیازمند داده بهتر
پیچیدگی بالا	مدیریت داده بهتر

جمع‌بندی

Data-Centric AI یکی از مهم‌ترین رویکردهای مدرن در یادگیری ماشین و MLOps است. این رویکرد بیان می‌کند که به جای تمرکز صرف بر توسعه مدل‌های پیچیده‌تر، باید کیفیت داده‌های آموزشی را بهبود داد. حذف داده‌های پرت، اصلاح برچسب‌ها، پاک‌سازی داده‌ها و تولید نمونه‌های مصنوعی از مهم‌ترین تکنیک‌های این حوزه هستند. در پروژه حاضر نیز با استفاده از مفاهیم Data-Centric AI، کیفیت داده‌های آموزشی افزایش یافته و زمینه لازم برای آموزش مدل‌های دقیق‌تر و پایدارتر فراهم شده است.

نکته ای شایان توجه که باید به آن توجه داشت این است که محتوای درج شده در این گزارش و سایر مستندات فنی پروژه، حاصل فرآیند تحقیق و توسعه (R&D) مستمر تیم فنی است. جهت تدوین این راهنماها، علاوه بر مستندات رسمی هر ابزار، از منابع چندگانه و متنوعی جهت بومی‌سازی و انطباق با نیازهای اختصاصی پروژه استفاده شده است. فرآیند جمع‌آوری دانش و تدوین این مستندات بر پایه محورهای زیر استوار بوده است:

۱. تحلیل منابع چندرسانه‌ای: بهره‌گیری از دوره‌های آموزشی تخصصی و سمینارهای فنی در پلتفرم‌های اشتراک ویدیو نظیر یوتیوب (YouTube) جهت درک عمیق پیاده‌سازی‌های عملی.

۲. جستجوهای تخصصی در وب: استخراج الگوهای بهینه (Best Practices) از طریق موتورهای جستجو و بررسی مقالات تحلیلی در وبسایت‌های مرجع مهندسی نرم‌افزار.

با توجه به گستردگی ابزارهای مورد استفاده در تیم فنی (Tech Stack) پروژه و حجم انبوه منابع بررسی شده، ذکر تمامی جزئیات، پیوندها (Links) و مراجع در این فایل و سایر اسناد مشابه میسر نبوده است. این مجموعه مستندات

صرفاً به عنوان یک «چهارچوب عملیاتی» و چکیده‌ای از یافته‌های کلیدی تدوین شده است تا فرآیند یادگیری اعضایی از تیم که با ابزارها آشنایی ندارند با سرعت و دقت بیشتری صورت پذیرد.

ممنون از توجه شما

تیم تحقیق

2. صورتجلسات

1-2 صورت جلسه هماهنگی پروژه MLOps (جلسه سوم - هفته دوم)

مشخصات جلسه:

تاریخ و زمان: 1405/03/07 - 60 دقیقه

پلتفرم برگزاری: جلسه آنلاین در پیام رسان بله

حاضرین: محمدرضا باقرنژادمیری، یاسمن دادفر، کیارش رحمانی، معصومه مختاری

۱. دستور جلسه

- مرور دستاوردهای هفته اول و ارزیابی وضعیت زیرساخت پروژه.
- بررسی الزامات هفته دوم مطابق سند پروژه MLOps.

- آشنایی با مفاهیم Experiment Tracking و Reproducible Pipeline.
- تصمیم‌گیری درباره رویکرد مدیریت چرخه داده و مدل.
- تقسیم وظایف مرتبط با ETL، MLflow و بهبود کیفیت داده‌ها.

۲. گزارش بخش مطالعاتی و منابع (مسئول: کیارش رحمانی)

مستندات مرجع

- مطالعه بخش «Pipelines, Reproducibility & Tracking» از سند پروژه MLOps.
- بررسی مفاهیم Stateful Training و Stateless Training و مقایسه مزایا و معایب هر رویکرد.
- مطالعه مقدماتی مستندات MLflow جهت ثبت آزمایش‌ها، پارامترها و مدل‌ها.
- بررسی منابع مرتبط با Data-Centric AI و نقش کیفیت داده در عملکرد مدل‌های یادگیری ماشین.

جمع‌بندی مطالعات

اعضای تیم به این نتیجه رسیدند که در فاز دوم پروژه تمرکز اصلی باید بر قابلیت بازتولید آزمایش‌ها (Reproducibility)، ثبت دقیق فرآیند آموزش مدل و بهبود کیفیت داده‌های آموزشی باشد. همچنین مقرر شد قبل از هرگونه توسعه مدل جدید، ابتدا زیرساخت ثبت آزمایش‌ها و مدیریت داده‌ها آماده شود تا تمامی تغییرات قابل رهگیری باشند.

خروجی کلیدی

ایجاد یک Pipeline استاندارد برای پردازش داده، آموزش مدل و ثبت نتایج در اولویت بالاتری نسبت به توسعه معماری‌های پیچیده مدل قرار گرفت.

۳. تصمیمات فنی جلسه

مدیریت آزمایش‌ها

پس از بررسی گزینه‌های موجود، استفاده از MLflow برای ثبت پارامترها، معیارهای ارزیابی، مدل‌ها و خروجی آزمایش‌ها مورد تأیید تیم قرار گرفت.

بهبود کیفیت داده‌ها

اعضا توافق کردند پیش از آموزش مجدد مدل، مجموعه داده از نظر موارد زیر بررسی شود:

- ❖ داده‌های پرت (Outlier)
- ❖ داده‌های تکراری
- ❖ خطاهای احتمالی برچسب‌گذاری
- ❖ امکان تولید نمونه‌های مصنوعی
- ❖ معماری Pipeline

تصمیم گرفته شد فرآیند آماده‌سازی داده و آموزش مدل به صورت ماژولار طراحی شود تا هر بخش مستقل از سایر بخش‌ها اجرا و آزمایش شود.

۴. وضعیت تسک‌ها (تسک‌بندی داوطلبانه)

- 1 مطالعه و مستندسازی MLflow
- 2 بررسی روش‌های Data Cleaning و Data Augmentation
- 3 طراحی اولیه ETL Pipeline
- 4 بررسی ساختار Model Registry در MLflow
- 5 تحلیل رویکرد Stateful و Stateless Training
- 6 تهیه مستندات فنی و ثبت نتایج مطالعات

۵. مصوبات و خروجی‌های جلسه

استفاده از MLflow به عنوان ابزار اصلی مدیریت آزمایش‌ها مورد تصویب قرار گرفت.

مقرر شد ساختار ETL Pipeline پیش از شروع آموزش‌های جدید نهایی شود.

اعضا موظف شدند تا جلسه آینده مستندات مربوط به وظایف خود را مطالعه و جمع‌بندی کنند.

تصمیم گرفته شد تمامی نتایج مطالعات، گزارش‌ها و مستندات فنی در گروه بله به اشتراک گذاشته شود.

جلسه بعدی به بررسی پیشرفت پیاده‌سازی MLflow و ETL اختصاص خواهد یافت.

تنظیم‌کننده گزارش: کیارش رحمانی

منابع مورد ارجاع: فایل راهنمای تمرین عملی MLOps + مستندات آنلاین استخراج شده.